



Haskell for Computational Linguists

Pranav Singh

`[first].[last]@ims.uni-stuttgart.de`

May 21, 2026

What are we doing today?

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

I will (gently) introduce you to a new paradigm, **functional programming**, and a new language, **Haskell**.

Agenda:

- 1 (10 min.) Introduction & installation
- 2 (15 min.) Recursion, thinking functionally, and Haskell syntax
- 3 (15 min.) Building a Finnish vowel harmonizer
- 4 (15 min.) Building an English → German VP translator



Who am I to tell you what to do?

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

Hello y'all :) My name is Pranav. I'm a fourth-semester M.Sc. student at Universität Stuttgart, studying Computational Linguistics (specializing in Digital Phonetics).

Work experience (boring):

- **2021 - 2023:** Product Manager @ New Relic
- **2017 - 2021:** B.A. in Linguistics & Computer Science @ University of California, Los Angeles
- **1999 - 2005:** My parents' favorite child

Coding:

- **before 2017:** Python, HTML
- **since 2017:** C/C++, Java, JavaScript, Go, Haskell, OCaml, Rust



Why care about Haskell?

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

- **Readable:** *Function composition* keeps code short and process-driven
- **Strong pattern-matching:** Easy to unpack and inspect custom data types
- **Elegant:** (My opinion, but) code looks clean and there's cute "syntactic sugar" everywhere

Specifically for linguists:

- **Recursion instead of iteration:** Easier to work with trees!
- **Actually used in our field:** For example, in my LING 185A: Computational Linguistics course at UCLA
 - Search "ling 185a ucla github" to find examples of previous student work, all in Haskell



What do you get out of it?

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

- A new language to add to your resume!
- Finally something that isn't Python
- Friendlier build tools like `stack` and `cabal`
 - Although `uv` has made Python environment management a lot easier ;)
- Very natural syntax compared to C++, Java, and other declarative languages



What do you need today?

Introduction

**Setup &
installation**

Hello.hs

Phonology.hs

Syntax.hs

- 1 Somewhere to write (VSCode, Zed...)
- 2 Somewhere to run (iTerm2, Terminal, etc...)
- 3 A free directory

If the installation does not work, you can use an online compiler, such as:

`play.haskell.org`



If you need to use an online compiler

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

While I will be demonstrating code running like this...

```
→ tacos $ ghci
GHCi, version 9.4.8: https://www.haskell.org/ghc/  :? for help
ghci> :l Hello
[1 of 1] Compiling Hello                ( Hello.hs, interpreted )
Ok, one module loaded.
ghci> square 5
25
ghci> factorialList [1, 2, 3, 4, 5]
[1,2,6,24,120]
ghci> 
```



If you need to use an online compiler

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

You will need to run things like this:



```
1 square :: Int -> Int
2 square x = x * x
3
4 factorial :: Int -> Int
5 factorial x
6   | x < 0    = error "(factorial) input cannot be negative"
7   | x == 0   = 1
8   | otherwise = x * (factorial (x - 1))
9
10 main :: IO ()
11 main = do
12   print (square 5)
13   print (factorial 5)
```

25
120



Installation & setup

Introduction

**Setup &
installation**

Hello.hs

Phonology.hs

Syntax.hs

- 1 haskell.org/ghcup/install
- 2 “How to install” → Download GHCup



Installation & setup

Introduction

**Setup &
installation**

Hello.hs

Phonology.hs

Syntax.hs

- 1 haskell.org/ghcup/install
- 2 “How to install” → Download GHCup
- 3 Copy everything from “...Stub.hs” files from my GitHub into your text editor
- 4 Edit the file directly
- 5 Test using `ghci` in your command line
- 6 Test often and test using edge cases! Let me know if you find something interesting :)



GHCi cheat sheet

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs



- 1 `$ ghci`
 - starts GHCi
- 2 `ghci> x = 5`
 - sets “x” to “5”
- 3 `ghci> x`
 - prints value of “x”
- 4 `ghci> y`
 - throws “Variable not found in scope” error
- 5 `ghci> :l Module`
 - loads file “Module.hs”
- 6 `ghci> :q`
 - quits GHCi

Let's try it!

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

- 1 Create a file named `Hello.hs`
- 2 Copy text from `HelloStub.hs` from my GitHub
- 3 Use `$ ghci` to test how you fare!

`github.com/pranavs99/tacos-workshop`



Finnish vowel harmony

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

How can we model the following phenomena?

- (1) *minä asu -n tä -ssä maa -ssa*
1.SG.NOM live -1.SG PROX -INESS country -INESS

“I live this-in country-in”

“I live in this country”

- (2) *hän muista -: tuo -sta kiele -stä*
3.SG.NOM remember -3.SG DIST -ELAT language -ELAT

“they remember-s that-from language-from”

“they (sg.) remember that language”



How does Finnish vowel harmony work?

Introduction

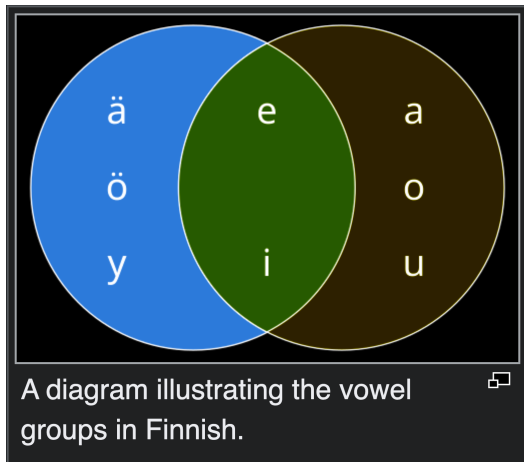
Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

blue = front, green = neutral, brown = back



How can we describe this as linguists?

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs



How can we describe this as linguists?

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

1 Affixes comes in two forms: **front** and **back**



How can we describe this as linguists?

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

- 1 Affixes comes in two forms: **front** and **back**
- 2 Vowels in affixes **match the frontness of the stem**



How can we describe this as linguists?

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

- 1 Affixes comes in two forms: **front** and **back**
- 2 Vowels in affixes **match the frontness of the stem**
- 3 When a stem only contains neutral vowels (“kiele”), the **front** affix form is used



How can we describe this as linguists?

Introduction

Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

- 1 Affixes comes in two forms: **front** and **back**
- 2 Vowels in affixes **match the frontness of the stem**
- 3 When a stem only contains neutral vowels (“kiele”), the **front** affix form is used
 - → front form is the **default**



German V2 movement

Introduction

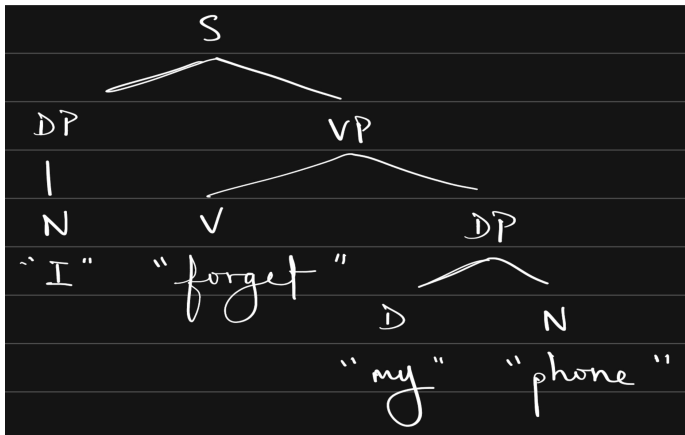
Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

"I forget my phone"



A hand-drawn syntax tree for the sentence "Ich vergesse mein Handy". The root node is "S", which branches into "DP" and "VP". The "DP" node branches into "I", which then branches into "N" with the word "Ich" written below it. The "VP" node branches into "V" with the word "vergesse" written below it, and another "VP" node. This second "VP" node branches into "DP", which then branches into "D" with the word "mein" written below it, and "N" with the word "Handy" written below it.

```
graph TD; S --> DP1[DP]; S --> VP1[VP]; DP1 --> I[I]; I --> N1[N]; N1 --> Ich["Ich"]; VP1 --> V[V]; V --> vergesse["vergesse"]; VP1 --> VP2[VP]; VP2 --> DP2[DP]; DP2 --> D[D]; D --> mein["mein"]; DP2 --> N2[N]; N2 --> Handy["Handy"];
```



German V2 movement

Introduction

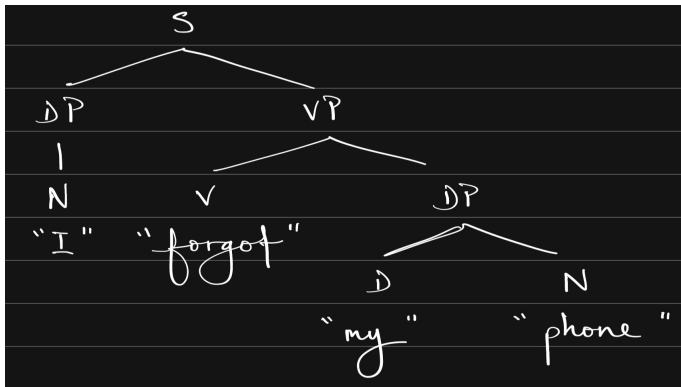
Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

"I forgot my phone"



German V2 movement

Introduction

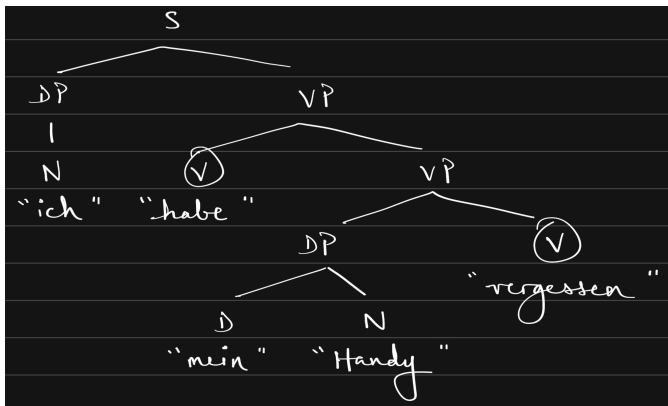
Setup &
installation

Hello.hs

Phonology.hs

Syntax.hs

“I forgot my phone” → “ich **habe** mein Handy **vergessen**”



Please rate the video, comment the video, like the video, share the video



Thank you! Any questions?

